

DynEDARP Project Description

DynEDARP: Dynamic Electric Dial-a-Ride Problem

A collaboration of Honda Research Institute Europe, TU Wien, and the Victoria University of Wellington.

Main aim is to develop and compare different heuristic solution approaches on DynEDARP, a dynamic dial-a-ride problem considering electric vehicles and their charging needs. The considered solving techniques include in particular Genetic Programming but also more traditional optimization-based methods.

In particular we want to obtain insight on the following aspects:

- Tradeoff between solution quality and computational time for different solution approaches
- Explainability of the solutions obtained by Genetic Programming and comparable learned heuristics

Potential Problem Definition

We consider the following variant of a Dynamic Electric Dial-a-Ride Problem (DynEDARP). The intention is to follow a classical DARP formulation and include the consideration of vehicle charging as well as dynamic aspects (online optimization), however, to keep things otherwise as simple as possible.

We are given a fleet $K = \{1, \dots, n_k\}$ of n_k electric (autonomous) vehicles (EAVs).

- Each vehicle has the same battery capacity Q ,
- an individual initial State-of-Charge (SoC) $0 < B_{k,1} \leq Q$,
- an initial location (i.e., node in the graph defined in the following),
- and a maximum load (max. number of persons to carry) C .

We are given a complete directed graph $G = (V, A)$ with node set V and arc set A .

- The node set V contains all somehow relevant locations in the considered geographical area.
- The arc set represents the fastest routes between all pairs of nodes.
 - For each arc $(i, j) \in A$, we are given a travel time t_{ij} and
 - a battery consumption β_{ij} that is for simplicity assumed to be proportional to the travel times.
 - It follows that the triangle inequality holds on G w.r.t. the travel times and battery consumption values.
- *GR: Or should we assume for simplicity arbitrary points in the plane and Euclidean distances? What do you think and how did you do this in your earlier work?*

There are in total n user requests $i = 1 \dots, n$ having

- pickup locations $P_i \in V$ and dropoff locations $D_i \in V$,
- time windows $[w_i^{\text{start}}, w_i^{\text{end}}]$ for the pickups,
- load values l_i (i.e., number of persons picked up or dropped off),
- and maximum ride times u_i between pickup and dropoff that must not be exceeded.
- Waiting is allowed when arriving at a location before the earliest start time w_i^{start} or when the vehicle becomes empty.
- We assume that service times at pickup and dropoff locations are included in the travel times between locations.

Graph G further contains a subset of n_S charging stations $S \subset V$, with $n_S = |S|$ being a small number, e.g., three.

- These charging stations have no time windows,
- and are able to handle arbitrary many vehicles also at the same time.
- Charging is assumed to be performed with a homogeneous charging rate α .
- Vehicles may only go to a charging stations when their current load is zero (i.e., no customers on board).
- At a charging station, we assume the vehicle always is fully charged. The time for charging vehicle k at its i -th stop at a charging station is then calculated as $(Q - B_{k,i})/\alpha$, where $B_{k,i}$ is the SoC when arriving at the charging station.

Dynamic aspects:

- User requests have times t_i^{avail} at which they become available, $i = 1, \dots, n$. Data of yet unavailable requests may not be used in the dynamic scenario. Initially available are all requests i with $t_i^{\text{avail}} = 0$.
- We assume requests arrive according to a Poisson process and we have information on the expected number of requests to arrive over time.
- Decisions for each vehicle of where to go next or if it has to wait at the current location are to be made
 - after arrival at a location when the vehicle is empty
 - and if the vehicle is empty waiting and a new request arrives.
 - As a side note, we assume here that all requests need to be fulfilled. In a practical situation, it is an additional decision problem to decide which incoming requests shall be accepted, and we do not focus on this aspect here.

Objective function to minimize:

- Total travel time over all routes (excluding service and charging times)
- plus a penalty for each request i that is served too late, i.e., where the pickup takes place after the end of the time window $\rho \cdot (P_i - w_i^{\text{end}})$, where ρ is a constant.

Benchmark instances:

- In our empirical evaluations, we will use randomly created instances with locations and distances in the Euclidean plane on the one hand, but on the other hand also consider a selection of instances from related real-world applications such as those available at https://github.com/aicenter/Ridesharing_DARP_instances.